INFORMATYKA / KLASA 2 / POZIOM: PODSTAWY → FUNKCJE

Python: misja Centrum wyników

Na dwóch lekcjach zbudujesz program analizujący punkty graczy. Nie kopiujesz gotowca: uruchamiasz przykłady, dopisujesz funkcje i testujesz ich działanie.

PLAN PRACY • ok. 2 × 45 minut

START (30 min): poznajesz i ćwiczysz funkcje. **PRO (35 min):** budujesz analizator wyników. **TESTY + BOSS (25 min):** sprawdzasz przypadki brzegowe i rozwijasz program.

01 Co to jest funkcja?

Funkcja to **nazwany fragment kodu**, który można wywoływać wiele razy. Może przyjmować dane (**parametry**) i oddawać wynik za pomocą **return**. Definicja funkcji zaczyna się od **def**.

```
def powitaj(nick):
    print("Witaj w turnieju,", nick)

powitaj("Ada")
powitaj("Kuba")
```

Mini-zadanie: uruchom kod, dodaj trzecie wywołanie i zmień tekst powitania. Co się stanie, jeśli usuniesz argument z wywołania **powitaj()**?

02 print() to nie to samo co return

```
def pokaz_bonus(punkty):
    print(punkty + 20)       # tylko wypisuje wynik

def policz_bonus(punkty):
    return punkty + 20      # oddaje wynik do programu

nowy_wynik = policz_bonus(100)
print(nowy_wynik * 2)      # 240
```

ZAPAMIĘTAJ

print() wyświetla tekst lub liczbę. **return** zwraca wartość, którą można zapisać w zmiennej, wykorzystać w obliczeniach lub przekazać do kolejnej funkcji.

ETAP START • OK. 20 MIN

Trzy małe funkcje = trzy zadania

Zapisz zadania w jednym pliku **funkcje_start.py**. Każdą funkcję wywołaj co najmniej dwa razy, używając innych argumentów.

ZADANIE 1. Premia punktowa

Napisz **dodaj_bonus(punkty, bonus)**, która **zwraca** sumę punktów i bonusu. Funkcja nie powinna sama nic wypisywać.

```
# Uzupełnij definicję funkcji:
def dodaj_bonus(punkty, bonus):
    pass

print(dodaj_bonus(120, 30))    # oczekiwane: 150
print(dodaj_bonus(85, 15))     # oczekiwane: 100
```

ZADANIE 2. Awans do finału

Napisz **czy_awans(punkty, prog)**, która zwraca **True**, gdy punkty są większe lub równe progowi; w przeciwnym razie **False**.

```
print(czy_awans(120, 120))  # True
print(czy_awans(119, 120))  # False
print(czy_awans(210, 200))  # True
```

ZADANIE 3. Ranga gracza

Napisz **ranga_gracza(punkty)**, która zwraca napis zgodnie z tabelą. Wykorzystaj **if / elif / else** i **return**.

Liczba punktów	Zwracany napis
poniżej 120	NOWICJUSZ
120–199	ZAWODNIK
200 i więcej	MISTRZ

Sprawdź: `ranga_gracza(119)` → NOWICJUSZ, `ranga_gracza(120)` → ZAWODNIK, `ranga_gracza(200)` → MISTRZ.

GDY NIE WIESZ, OD CZEGO ZACZĄĆ

Przy funkcji najpierw odpowiedz na 3 pytania: **(1)** jakie dane dostaje? **(2)** co ma obliczyć? **(3)** co ma zwrócić? Zapis **pass** jest tylko pustym miejscem – zamień go na własny kod.

ETAP PRO • OK. 35 MIN

Projekt: Centrum wyników

Teraz przygotujesz raport z turnieju. Użyj funkcji tak, aby każda odpowiadała za **jedno zadanie**. Nie zmieniaj listy wewnątrz funkcji.

Dane startowe — 7 wyników graczy

```
wyniki = [120, 85, 160, 85, 210, 95, 140]
```

Dopisz 4 funkcje:

- **najlepszy_wynik(lista)** – zwraca największy wynik. Możesz użyć `max()`.
- **policz_srednia(lista)** – zwraca średnią arytmetyczną, zaokrągloną do 2 miejsc. Przydadzą się `sum()`, `len()` i `round()`.
- **policz_awansy(lista, prog)** – zwraca liczbę wyników **większych lub równych** progowi. Tu zastosuj pętlę i licznik.
- **pokaz_raport(lista)** – wywołuje powyższe funkcje i wyświetla raport. Użyj progu **120 pkt**.

Szkielet programu – możesz od niego zacząć

```
wyniki = [120, 85, 160, 85, 210, 95, 140]

def najlepszy_wynik(lista):
    pass

def policz_srednia(lista):
    pass

def policz_awansy(lista, prog):
    pass

def pokaz_raport(lista):
    pass

pokaz_raport(wyniki)
```

Oczekiwane działanie:

```
=== CENTRUM WYNIKOW ===
Najlepszy wynik: 210
Srednia punktow: 127.86
Liczba awansow (prog 120): 4
```

WARTO WIEDZIEĆ

Funkcja **pokaz_raport()** służy do wyświetlania, więc może używać **print()**. Funkcje, które **obliczają** wynik, powinny go zwracać przez **return**. To ułatwia ponowne wykorzystywanie kodu.

ETAP TESTY + BOSS • OK. 25 MIN

Nie wystarczy, że program działa raz

Zanim oddasz pracę, sprawdź, czy działa również dla innych danych. Programista testuje granice, nie tylko najłatwiejsze przykłady.

TEST 1. Zmień dane – nie zmieniaj funkcji

```
inne_wyniki = [200, 100, 120, 80]
pokaz_raport(inne_wyniki)
```

Oczekiwane: rekord **200**, średnia **125.0**, awansuje **2** graczy. Jeśli musisz przepisywać kod funkcji, zastanów się, czy rzeczywiście korzysta z parametrów.

TEST 2. Sprawdź granicę awansu

Czy wynik **120** kwalifikuje do finału? Sprawdź `policz_awansy([119, 120, 121], 120)`. Poprawny rezultat to **2**.

TEST 3. Znajdź błąd: print czy return?

```
def srednia(lista):
    print(sum(lista) / len(lista))

x = srednia([10, 20, 30])
print(x)
```

Co wypisze program? Dlaczego? Jak zmienić funkcję, aby x przechowywał liczbę 20?

BOSS • DLA OSÓB, KTÓRE SKOŃCZĄ WCZEŚNIEJ

A. Dodaj funkcję `top3(lista)`, która zwraca 3 największe wyniki od najwyższego. **B.** Obsłuż pustą listę w `policz_srednia([])` – ustal, co funkcja powinna wtedy zwrócić, aby program nie uległ awarii. **C.** Przenieś funkcje do osobnego pliku i zaimportuj je w pliku głównym.

Lista kontrolna przed oddaniem

☐ Umiem zdefiniować i wywołać funkcję z parametrami.

☐ Rozumiem różnicę między print() a return.

☐ Moje funkcje działają także dla innej listy wyników.

☐ Potrafię wskazać funkcję, która wywołuje inne funkcje.

Oddajesz: jeden plik `.py` z funkcjami, wywołaniami testowymi i krótkim komentarzem przy każdej funkcji, opisującym jej zadanie.